Reviewer Pack — Erdős–Koós Inequality and ACC⁰ Circuit Depth

Entry 15ec19309767 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 01:01:19 UTC

Erdős–Koós Inequality and ACC⁰ Circuit Depth

Entry ID: `15ec19309767` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 01:01:19 UTC

1. Conjecture

Field A (mathematical branch): Additive Combinatorics **Field B** (complexity object): Boolean Circuit Complexity

Statement:

> ['For every explicit function f in P computed by an ACC⁰ circuit with depth D, the number of edges in the associated incidence graph G_f satisfies E[|E(G_f)|] ≤ α * √(nD), where n is the input size and α is a constant.', 'Equivalently, for any explicit function f in P, there exists a constant β such that if an ACC⁰ circuit computes f with depth D, then E[|E(G_f)|/√(nD)] ≤ β.']

Rationale (proposer's reasoning):

> ['The Erdős–Koós inequality provides a strong bound on the number of edges in the incidence graph for certain types of polynomials. If this inequality holds for ACC⁰ circuits, it could lead to nontrivial lower bounds for ACC⁰ by relating the circuit depth to the complexity of the incidence graph.', 'This bridge would expose a new way to understand the structure of ACC⁰ circuits and potentially lead to insights into the nature of polynomial-time computation.']

Taxonomy category: `ACC_SIPSER` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `3762420b1fa4ef7e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For an ACC⁰ circuit computing a function f in P with depth D, if E[|E(G_f)|/√(nD)] ≤ β for all seeds and the mean value of |E(G_f)|/√(nD) is below a threshold T, the conjecture is supported. If any seed produces |E(G_f)|/√(nD) > β + T or any seed produces |E(G_f)|/√(nD) > 10, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - additive combinatorics AND boolean circuit complexity AND Erdős–Koós inequality - ACC⁰ circuit depth AND incidence graph AND α * √(nD) - Boolean circuits AND ACC⁰ AND E[|E(G_f)|/√(nD)] ≤ β

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Define constants and parameters
    n = 30  # Input size
    depth = random.randint(5, 40)  # Depth of ACC⁰ circuit
    num_trials = 100  # Number of instances to test

    total_edges = 0

    for _ in range(num_trials):
        # Generate a random polynomial f(x1, ..., xn)
        coefficients = [random.randint(0, 1) for _ in range(n)]
        f = sum(c * x**i for i, c in enumerate(coefficients))

        # Construct the incidence graph G_f
        edges = set()
        for i in range(n):
            for j in range(i + 1, n):
                if f.subs({x: i}) == f.subs({x: j}):
                    edges.add((i, j))

        total_edges += len(edges)

    avg_edges_per_trial = Fraction(total_edges, num_trials)
    metric_value = avg_edges_per_trial / math.sqrt(n * depth)
```

```
    # Define the acceptance criterion
    beta = 1.0  # Example constant
    T = 0.1  # Example threshold

    conjecture_holds = metric_value <= beta + T and metric_value <= 10
    counterexample = "" if conjecture_holds else "beta_threshold_exceeded"

    return {
        "metric_name": "avg_edges_per_trial",
        "metric_value": avg_edges_per_trial,
        "instances_tested": num_trials,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys

    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"beta_threshold_exceeded\" first_failing_seed={f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8227ac7a.py", line 67, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8227ac7a.py", line 31, in run_trial
    f = sum(c * x**i for i, c in enumerate(coefficients))
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8227ac7a.py", line 31, in <genexpr>
    f = sum(c * x**i for i, c in enumerate(coefficients))
```

```
                ^
NameError: name 'x' is not defined
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means that it was unable to complete the computation necessary to evaluate the conjecture. | next: Investigate and fix the crash in the test code to allow for a proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11587
2	preregistration	ollama_remote	glm4:latest	0	0	6789
3	novelty	ollama_remote	glm4:latest	0	0	4840
4	novelty	ollama_remote	glm4:latest	0	0	5878
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19593
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12425
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17802
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8064
9	judge	ollama_remote	glm4:latest	0	0	10507

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 97485 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/15ec19309767.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/15ec19309767.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/15ec19309767.tar.gz` (if generated)